

Laboratório
Concorrente

de

Programação

Lab10 - 26.1

Concorrência em Go
com Goroutines e
Canais

Objetivos

Após este roteiro prático, você deverá ser capaz de:

- compreender os fundamentos do modelo CSP (*Communicating Sequential Processes*);
- criar goroutines;
- utilizar canais para comunicação entre goroutines;
- compreender o funcionamento de canais bufferizados e não bufferizados;
- utilizar canais unidirecionais;
- construir pipelines concorrentes em Go

Contexto

Até então, estudamos concorrência utilizando **memória compartilhada**, através de mecanismos como:

- Mutexes
- Semáforos
- Monitores
- Variáveis condicionais

Na linguagem Go, a abordagem predominante é diferente. Em vez de compartilhar a memória e sincronizar o acesso a ela, as rotinas trocam mensagens por meio de **canais**. Essa abordagem é baseada no modelo **Communicating Sequential Processes (CSP)**, proposto por Tony Hoare.

A filosofia da linguagem é resumida pela frase:

Don't communicate by sharing memory; share memory by communicating.

Ao longo deste roteiro construiremos uma aplicação concorrente de forma incremental.

Vocês devem criar os arquivos mencionados ao lado de cada exemplos, e, ao final, entregar a versão final de todos os arquivos solicitados.

Primeiro programa em Go (main1.go)

Crie um arquivo, ([main1.go](#)) e digite o código abaixo.

```
package main

import "fmt"

func main() {
    fmt.Println("Olá Go!")
}
```

Para executar o programa, você deve usar o comando:

```
go run main.go
```

Primeira goroutine (main2.go)

Nesta etapa vamos observar a diferença entre uma chamada sequencial de função e uma chamada concorrente utilizando goroutines.

1 – Execução sequencial

Implemente o código abaixo.

```
package main

import "fmt"

func mensagem() {
    fmt.Println("Executando mensagem")
}

func main() {

    mensagem()

    fmt.Println("Fim da main")
}
```

Execute o programa e observe o seguinte:

Qual mensagem apareceu primeiro?

A saída sempre é a mesma?

Como a função foi chamada normalmente, a execução ocorre de forma **sequencial**.

2 – Transformando em uma goroutine

Agora altere apenas uma linha. Adicione a palavra reservada `go` logo antes da chamada da função `mensagem()`.

Execute o programa várias vezes e observe o seguinte:

A mensagem sempre aparece?

A ordem das mensagens permanece a mesma?

3 – Discussão inicial

Ao utilizar `"go mensagem()"` uma nova goroutine é criada. A função `main()` não espera essa goroutine terminar. Quando `main()` finaliza, todo o programa é encerrado. Por isso, algumas execuções podem produzir apenas: Fim da main

4 – Retardando o final da main

Agora faremos a função principal esperar um pouco.

```
package main

import (
    "fmt"
    "time"
)

func mensagem() {
    fmt.Println("Executando mensagem")
}

func main() {
    go mensagem()
    time.Sleep(time.Second)
    fmt.Println("Fim da main")
}
```

5 - Discussão

O uso de `time.Sleep()` não é um mecanismo de sincronização. Ele apenas atrasa a execução da função principal. Mais adiante veremos como utilizar canais para sincronização adequada.

Introdução aos canais (main3.go)

Até agora as goroutines executam independentemente. Mas como elas podem trocar informações? Em Go isso ocorre através dos **canais** (**channels**). Um canal conecta duas ou mais goroutines. Uma envia dados. Outra recebe. Além da comunicação, o canal também realiza sincronização.

Criando um canal

```
canal := make(chan string)
```

O canal poderá transportar apenas valores do tipo `string`.

Enviando dados

```
canal <- "Olá"
```

Recebendo dados

```
texto := <-canal
```

Primeiro exemplo

```
package main

import "fmt"

func worker(c chan string) {
    c <- "Olá da goroutine"
}

func main() {
    canal := make(chan string)
    go worker(canal)
    mensagem := <-canal
    fmt.Println(mensagem)
}
```

Discussão

Execute o programa e observe o seguinte:

- Quem envia?
- Quem recebe?
- Quem espera quem?

Bloqueio automático

Os canais criados com

`make(chan T)`

são chamados de **canais não bufferizados**.

Neles:

- quem envia espera alguém receber;
- quem recebe espera alguém enviar.

Isso gera sincronização automática entre as goroutines.

Canais bufferizados

Nem sempre queremos que produtor e consumidor executem exatamente no mesmo instante. Go permite criar canais com buffer.

Exemplo:

```
canal := make(chan int, 3)
```

Esse canal consegue armazenar até **3 valores** antes que o produtor seja bloqueado.

Exemplo:

```
canal <- 10  
canal <- 20  
canal <- 30
```

Todos esses envios acontecem imediatamente.

O quarto envio ficará bloqueado até que algum valor seja consumido.

Discussão

Pense um pouco sobre:

- Qual a vantagem de um canal bufferizado?
- Em quais situações ele reduz o tempo de espera entre produtor e consumidor?

Canais unidirecionais

Em muitos casos uma função deve apenas enviar ou apenas receber mensagens. Go permite restringir essa utilização.

Canal somente para envio

```
func produtor(c chan<- int)
```

A função só pode enviar. Qualquer tentativa de leitura gera erro de compilação.

Canal somente para recebimento

```
func consumidor(c <-chan int)
```

A função apenas recebe. Não pode enviar valores.

Discussão

Essas restrições:

- tornam o código mais legível;
- evitam erros de programação;
- deixam explícito o papel de cada goroutine.

Enviando vários valores

Agora modifique o produtor para enviar cinco números.

```
func produtor(c chan<- int) {  
    for i := 1; i <= 5; i++ {  
        c <- i  
    }  
}
```

Na função principal:

```
for i := 1; i <= 5; i++ {  
    fmt.Println(<-canal)  
}
```

Discussão

Execute o programa e observe que produtor e consumidor permanecem sincronizados automaticamente.

Dois produtores (main4.go)

Agora duas goroutines compartilharão o mesmo canal.

```
func produtor(nome string, c chan<- string) {
    for i := 1; i <= 5; i++ {
        c <- fmt.Sprintf("%s -> %d", nome, i)
        time.Sleep(500 * time.Millisecond)
    }
}
```

Na função principal:

```
canal := make(chan string)
go produtor("A", canal)
go produtor("B", canal)

for i := 0; i < 10; i++ {
    fmt.Println(<-canal)
}
```

Discussão

Execute diversas vezes e observe que a ordem das mensagens muda a cada execução.

Fechando canais

Quando um produtor termina definitivamente seu trabalho, ele pode fechar o canal.

```
close(canal)
```

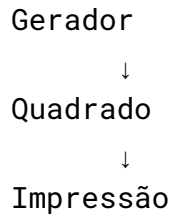
Consumindo:

```
for valor := range canal {
    fmt.Println(valor)
}
```

O comando **range** continua lendo enquanto houver dados disponíveis. Quando o canal é fechado e esvaziado, o laço termina automaticamente.

Pipeline concorrente (main5.go)

Construa o pipeline:



Segue o código:

```
package main
import "fmt"

func geraNumeros(c chan int) {
    for i := 1; i <= 5; i++ {
        c <- i
    }
    close(c)
}

func quadrado(entrada chan int, saida chan int) {
    for n := range entrada {
        saida <- n * n
    }
    close(saida)
}

func main() {
    entrada := make(chan int)
    saida := make(chan int)

    go geraNumeros(entrada)
    go quadrado(entrada, saida)

    for resultado := range saida {
        fmt.Println(resultado)
    }
}
```

Discussão

Execute o código e observe que cada etapa executará em sua própria goroutine de forma sincronizada.

Exercício do Lab (main6.go)

Desenvolva um sistema de monitoramento composto por:

- dois sensores de temperatura (produtores);
- um canal compartilhado;
- um consumidor responsável por processar as leituras.

Cada sensor deverá:

- gerar cem temperaturas aleatórias entre 0 e 60;
- aguardar um intervalo entre as medições;
- enviar os dados ao consumidor.

O consumidor deverá calcular:

- quantidade de medições;
- temperatura média;
- maior temperatura;
- menor temperatura.

Entrega

A entrega se dará através das atualizações do código no próprio repositório no Github Classroom. Você deve submeter os seis arquivos main mencionados anteriormente.

Exemplos

Gerar Número Aleatório em Go

```
import (  
    "fmt"  
    "math/rand"  
    "time"  
)  
  
func main() {  
    // A partir do Go 1.20, a semente (seed) é inicializada  
    // automaticamente.  
    // Em versões anteriores, era necessário usar  
    rand.Seed(time.Now().UnixNano())  
  
    // Gera um número inteiro de 0 até 99  
    numero := rand.Intn(100)  
    fmt.Println("Número aleatório:", numero)  
}
```